

**Documentazione per il progetto di
Programmazione Avanzata, UniPi**

MakerManager

**Sistema di gestione dell'inventario e di
pianificazione dei progetti**

Giuseppe Vaglica, matr. 672516

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025/2026

Sommario

Sommario.....	1
1. Caso d'uso.....	2
1.1. Funzioni per l'utente	2
2. Struttura del Progetto	3
2.1. Architettura del Sistema	3
2.2. Client	3
2.2.1. Gestione della GUI	3
2.2.2. Dettaglio sulle schermate.....	3
2.3. Server	4
2.3.1. Sicurezza e autenticazione.....	5
2.3.2. Modello Dati.....	5
2.3.3. Logica di servizio	5
2.3.4. API.....	6
3. Scelte Progettuali	6
4. Test e utilizzo del sistema	7
5. Interfaccia e esempi di utilizzo	8
5.1. Pannelli principali	9
5.1.2. Dashboard.....	9
5.1.3. Tutti i Progetti.....	9
5.1.4. Inventario	10
5.1.5. I Miei Progetti.....	10
5.1.6. Database (utente / admin)	11
5.1.7. Impostazioni	11
5.2. Gestione dell'inventario	11
5.2.1. Visualizzazione di un proprio inventario.....	12
5.2.2. Aggiunta di un articolo all'inventario	12
5.2. Gestione dei progetti	12
5.2.1. Visualizzazione di un progetto	13
5.2.2. Aggiunta di un articolo alla BOM del proprio progetto	13
6. Sviluppi futuri	14
7. Note sull'uso dell'Intelligenza Artificiale	15

1. Caso d'uso

Chi lavora come maker (in senso generico un maker è un creatore, o un piccolo artigiano, ma nel contesto contemporaneo un maker è una persona creativa che coltiva l'hobby e produce oggetti di elettronica, stampa 3D, robotica, piccola prototipazione, ecc.) accumula nel tempo una quantità disordinata di componenti: resistori, sensori, bobine di filamento, moduli, attrezzi. Tre problemi si presentano di continuo:

1. non si sa più **cosa si ha**, e si finisce per ricomprare pezzi già posseduti;
2. non si sa **cosa farci**: si vorrebbe partire con un progetto nuovo, ma capire quale progetto sia realizzabile con il materiale già in casa richiede di confrontare a mano la distinta base di ogni progetto con il proprio magazzino.

Alla luce di questi problemi ho scelto il caso d'uso per questo progetto: sviluppare un gestionale per maker con l'obiettivo di tenere traccia dei progetti caricati dagli utenti, tenere traccia delle proprie disponibilità e avere a disposizione una serie di funzionalità di comodo per gli utenti. Tra queste spicca soprattutto la possibilità di cercare nuovi progetti a cui dedicare del tempo, scegliendo specificatamente i progetti che sono coerenti con le proprie disponibilità di risorse nell'inventario.

In questo modo, il maker può sia esplorare i progetti da un catalogo pubblico dove gli utenti caricano progetti divisi per tipologia, e può sia visualizzare nella sua Dashboard i progetti affini al suo inventario. Inoltre, l'utente stesso è libero di contribuire al catalogo dei progetti.

1.1. Funzioni per l'utente

Sinteticamente l'utente può accedere alle seguenti funzionalità:

- Autenticarsi o registrarsi nel sistema.
- Visualizzare una dashboard con le informazioni generali e i progetti consigliati da MakerManager.
- Visualizzare l'elenco pubblico di progetti caricati dagli utenti.
- Accedere e gestire il suo o i suoi inventari.
- Visualizzare e modificare il contenuto di un suo inventario.
- Aggiungere nuovi articoli all'inventario importandoli dagli elementi condivisi nel database comune oppure creando nuovi elementi e importandoli nell'inventario.
- Visualizzare, creare, eliminare e modificare i propri progetti. Dopo aver creato un progetto, l'utente proprietario può modificare la sua BOM (*Bill Of Materials*).
- Visualizzare da un database pubblico gli elementi presenti nel catalogo, e crearli.
- Modificare dalle impostazioni il parametro di soglia per cui vengono consigliati i progetti nella dashboard.

2. Struttura del Progetto

2.1. Architettura del Sistema

Il Sistema presenta una struttura di tipo Client – Server, dove la comunicazione tra client e server avviene esclusivamente attraverso chiamate di tipo GET, POST, DELETE e PATCH, alle API REST esposte dal Server stesso, che a sua volta gestisce la comunicazione con il database MySQL, attraverso JPA/Hibernate. Il Client non comunica mai direttamente con il database. Tutte le informazioni sono scambiate in formato JSON.

2.2. Client

2.2.1. Gestione della GUI

Il Client è organizzato in **file FXML** (struttura delle schermate), **controller** (funzionalità), **service** (aree del dominio) e una classe **ApiClient** dedicata esclusivamente alla comunicazione HTTP. Lo stile è invece centralizzato in un unico *style.css* applicato a ogni pannello, popup e finestra.

2.2.2. Dettaglio sulle schermate

All'avvio compare la schermata di **autenticazione**; solo dopo il login la root della scena viene sostituita con la **Shell** (sidebar di navigazione + area contenuti). I sei pannelli sono precaricati una sola volta e ricaricati ogni volta che tornano visibili.

Schermata	Descrizione
Dashboard	Stato del server, statistiche e tabella dei progetti consigliati.
Tutti i Progetti	Catalogo condiviso; cliccando su un progetto viene aperta la finestra di dettaglio con la B.O.M.
Inventario	Elenco dei propri inventari; il click sull'inventario mostra il contenuto, con aggiunta, eliminazione e modifica della quantità.
I Miei Progetti	Progetti creati dall'utente; creazione, eliminazione e gestione della B.O.M.
Database	Catalogo elementi; per l'ADMIN, pulsante Inizializza ed eliminazione elementi.
Impostazioni	Impostazione della soglia dei progetti consigliati.

Due comportamenti ricorrenti:

- **Eliminazione da una lista:** il pulsante *Elimina* attiva la **modalità eliminazione**, il click su una riga apre un popup di conferma e solo dopo l'OK parte la DELETE.
- **Aggiunta di un articolo o di una riga B.O.M.:** il popup permette di *importare* un elemento già in catalogo oppure di *crearne uno nuovo*, che viene aggiunto contestualmente al catalogo condiviso e alla destinazione.

2.3. Server

2.3.1. Sicurezza e autenticazione

L'autenticazione è gestita con Spring Security. In fase di registrazione servono nickname, e-mail e password; quest'ultima non è mai salvata in chiaro ma cifrata con BCrypt. Login e registrazione restituiscono un token JWT, che il Client allega a ogni richiesta successiva nell'header.

Il sistema è multiruolo (UTENTE e ADMIN; l'ADMIN di default è creato all'avvio da AdminBootstrap). I permessi di accesso agli endpoint non sono scritti in Java, ma in un file esterno ricaricato a cadenza regolare di 5s (config/permessi-endpoint.properties). I permessi di proprietà sulle entità non dipendono dall'URL e sono quindi verificati nei service tramite UtenteCorrente: il proprietario di una risorsa non è mai indicato dal Client, ma sempre ricavato dal token.

2.3.2. Modello Dati

I modelli dati associati alle entità sono stati sviluppati in previsione di potenziali future espansioni del sistema e nell'ottica di aggiungere nuove funzionalità. Sono di seguito elencati i principali modelli dati:

- **Utente**: account registrato, con ruolo UTENTE o ADMIN.
- **ElementoCatalogo**: il modello teorico di un oggetto.
- **Inventario**: contenitore di articoli appartenente a un utente.
- **ArticoloInventario** (astratta): il possesso fisico di un ElementoCatalogo in un Inventario, con la relativa quantità. Sottoclassi: *ComponenteElettronico*, *MaterialeConsumabile*, *AttrezzoDaLavoro*, *Software*.
- **ProgettoMaker** (astratta): progetto del catalogo condiviso, con autore, tipologia, distinta base e progresso. Sottoclassi: *ProgettoStampa3D*, *ProgettoElettronica*, *ProgettoRobotica*, *ProgettoSoftware*.
- **BOM** (@Embeddable) — la distinta base, contenitore di RigaBOM.
- **RigaBOM** — singolo requisito: "servono N unità di questo ElementoCatalogo".
- **Progresso** (@Embeddable) — stato di avanzamento del progetto.

2.3.3. Logica di servizio

La logica è suddivisa in service, uno per area:

- **InizializzazioneService**: /inizializza: cancella i dati esistenti e li ricarica da *inizializzazione.json*, in un'unica transazione.
- **AuthService**: registrazione e login (BCrypt + JWT).
- **UtenteService, CatalogoService**: profili utente e catalogo elementi.
- **InventarioService, ProgettoService**: gestione di inventari, articoli, progetti e righe B.O.M., con verifica di proprietà.
- **ProgettiConsigliatiService**: aggrega le disponibilità dell'utente, esclude i progetti propri e quelli con B.O.M. vuota, valuta la fattibilità dei restanti, filtra per soglia e ordina.

A supporto: *CalcolatoreFattibilita*, che contiene il calcolo vero e proprio e non dipende da database, sicurezza o Spring (ed è per questo facilmente testabile tramite gli JUnit Test); i mapper (da entity a DTO e viceversa); le factory, che istanziano la sottoclasse corretta a partire dalla tipologia; il *GlobalExceptionHandler*, che traduce le eccezioni di dominio in risposte HTTP uniformi e controllate.

2.3.4. API

La documentazione API è piuttosto vasta, ricca di esempi con comandi curl da terminale ed è posta in un file separato. È possibile consultare la documentazione in **docs/endpoint.md**.

3. Scelte Progettuali

Tra le scelte progettuali rientrano sicuramente:

- Utilizzo di Spring Boot per il Server
- Applicazione ferrea di S.R.P. (single responsibility principle)
- Utilizzo combinato di Enum + Factory + Classe abstract per *ArticoloInventario* e per *ProgettoMaker*
- Gestione dei dati scambiati tra client e server mediante DTO e classi Mapper.
- Utilizzo di un *GlobalExceptionHandler* (in accordo con SRP) per una gestione controllata delle eccezioni.
- Sicurezza HTTP mediante Spring Boot Security e token JWT.
- Sviluppo mirato a espansioni future del software.
- Gestione dei permessi facilmente configurabile da *permessi-endpoint.properties* senza necessità di riavvio del server.

4. Test e utilizzo del sistema

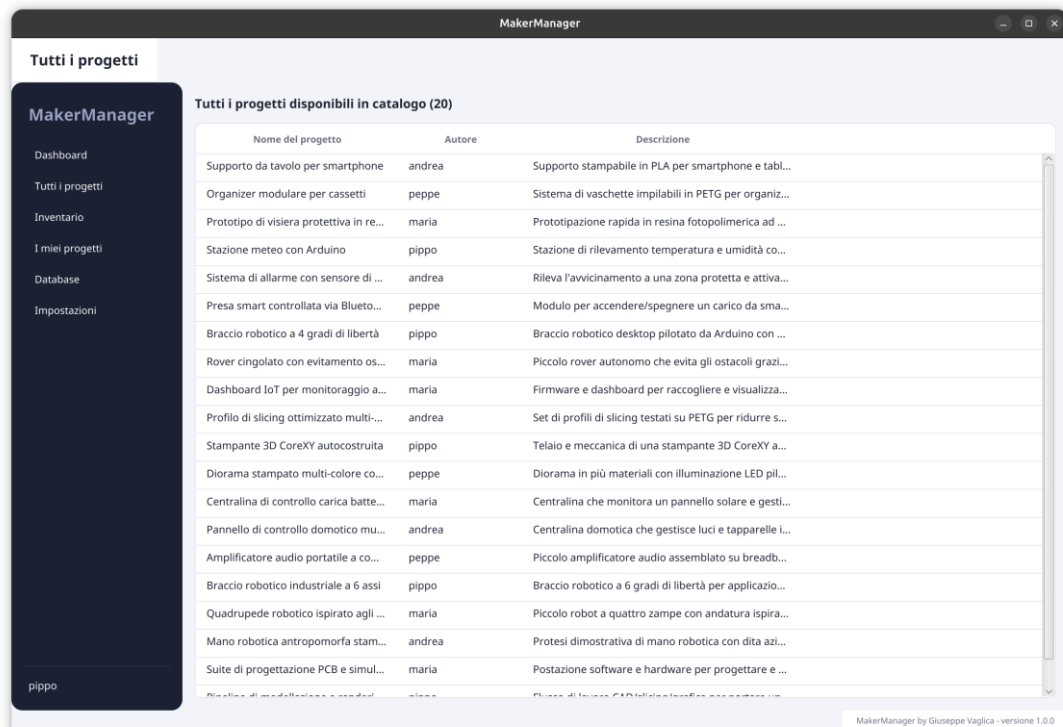
All'interno del sistema sono presenti sia dei test automatici (6 JUnit test che coprono tutto il calcolo della fattibilità di un progetto, testando la classe core del caso d'uso centrale per consigliare progetti agli utenti), sia tutta una serie di test che coprono le chiamate http a tutti gli endpoint delle API, includendo casi limite e casi vietati. I primi si trovano nella cartella *src/test/java/it/unipi/makermanagerserver/manager/raccomandazioni*, mentre gli ultimi si trovano in *src/test/chiamate-endpoint/http*, qui è presente anche un piccolo manuale d'uso.

Per l'utilizzo del sistema è possibile autenticarsi con uno degli account di test:

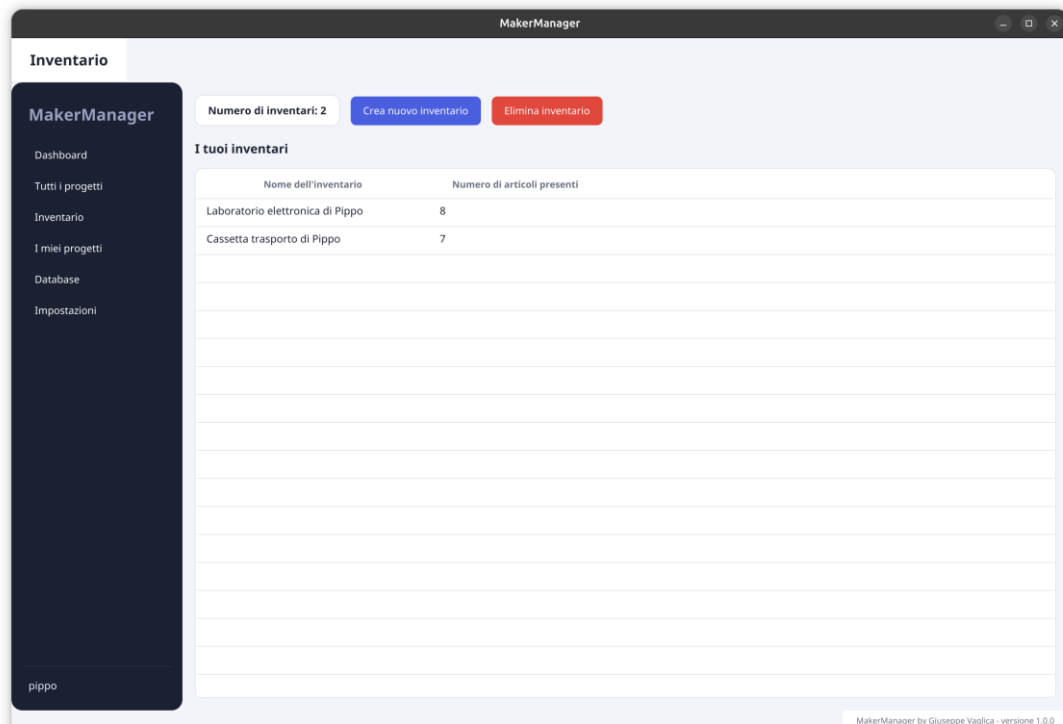
Nickname	Email	Password	Ruolo
giuse27	giuse27@makermanager.com	LOUVRE2014	ADMIN
pippo	pippo@test.com	LOUVRE2014	UTENTE
maria	maria@test.com	LOUVRE2014	UTENTE
peppe	peppe@test.com	LOUVRE2014	UTENTE
andrea	andrea@test.com	LOUVRE2014	UTENTE

NOTA BENE: al primo utilizzo del sistema è necessario autenticarsi con l'account ADMIN di giuse27@makermanager.com. Solo l'account con ruolo ADMIN ha il permesso di inizializzare il sistema. La sua presenza è sempre garantita a differenza degli altri utenti che non esistono se il database è vuoto.

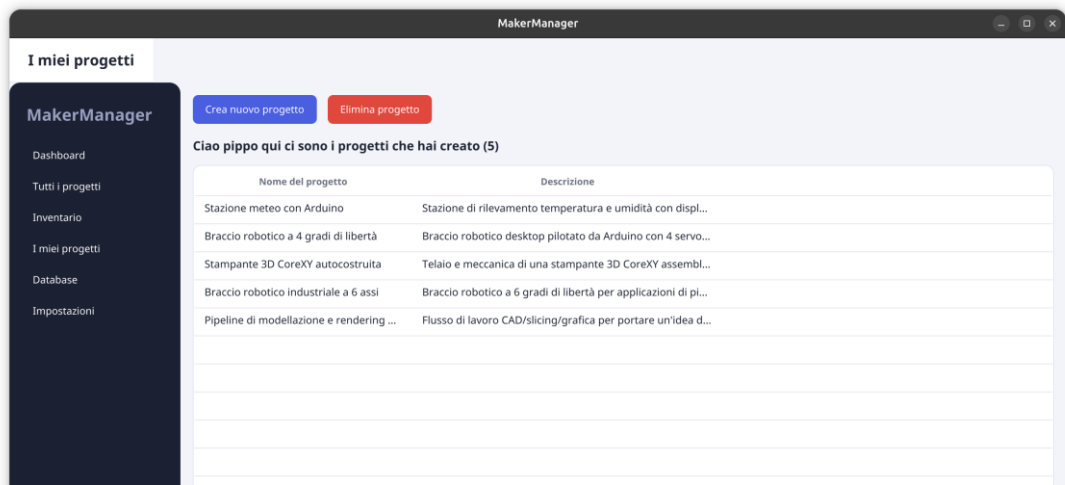
5.1.3. Tutti i Progetti



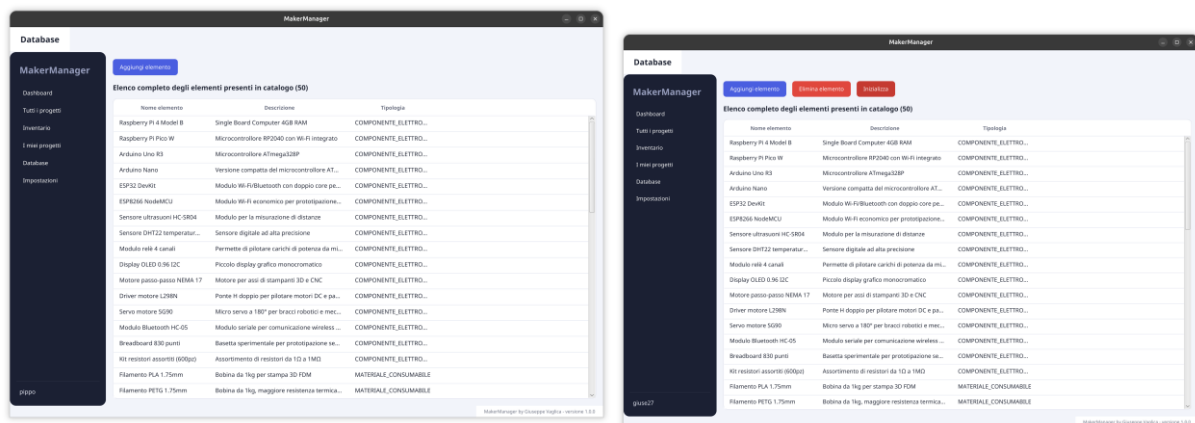
5.1.4. Inventario



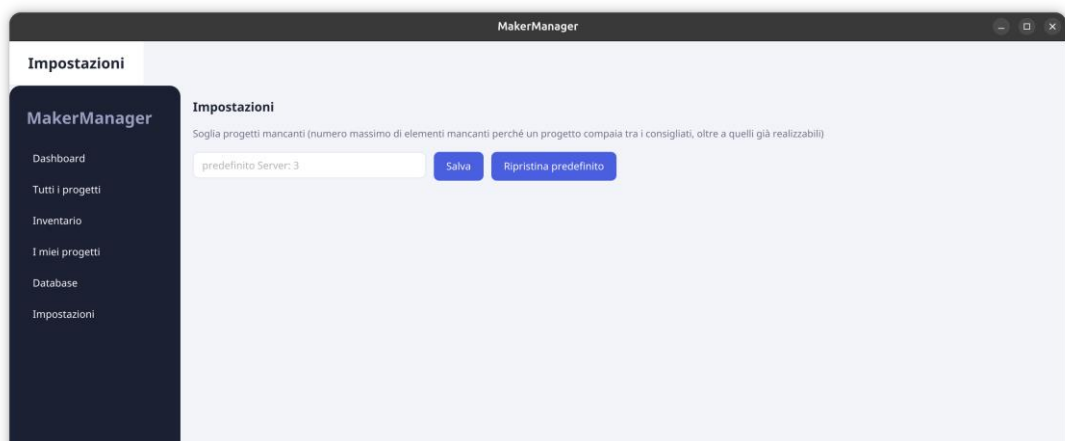
5.1.5. I Miei Progetti



5.1.6. Database (utente / admin)



5.1.7. Impostazioni



5.2. Gestione dell'inventario

5.2.1. Visualizzazione di un proprio inventario

Laboratorio elettronica di Pippo

Numero articoli: 8 [Aggiungi un nuovo articolo](#) [Elimina un articolo](#)

Laboratorio elettronica di Pippo

Nome articolo	Quantità
Arduino Uno R3	3
ESP32 DevKit	2
Sensore ultrasuoni HC-SR04	4
Sensore DHT22 temperatura/umidità	2
Display OLED 0.96 I2C	2
Breadboard 830 punti	3
Kit resistori assortiti (600pz)	1
Driver motore L298N	2

5.2.2. Aggiunta di un articolo all'inventario

Aggiungi articolo a "Laboratorio elettronica di ..."

[Importa dal Catalogo](#) [Crea nuovo elemento](#)

Elemento del Catalogo

Raspberry Pi Pico W (COMPONENTE_ELETTRONICO)

Quantità

3

[Aggiungi all'inventario](#)

Aggiungi articolo a "Laboratorio elettronica di ..."

[Importa dal Catalogo](#) [Crea nuovo elemento](#)

Nome

Nuovo elemento

Descrizione

Test di aggiunta di un nuovo elemento al catalogo per l'inventario

Tipologia

MATERIALE_CONSUMABILE

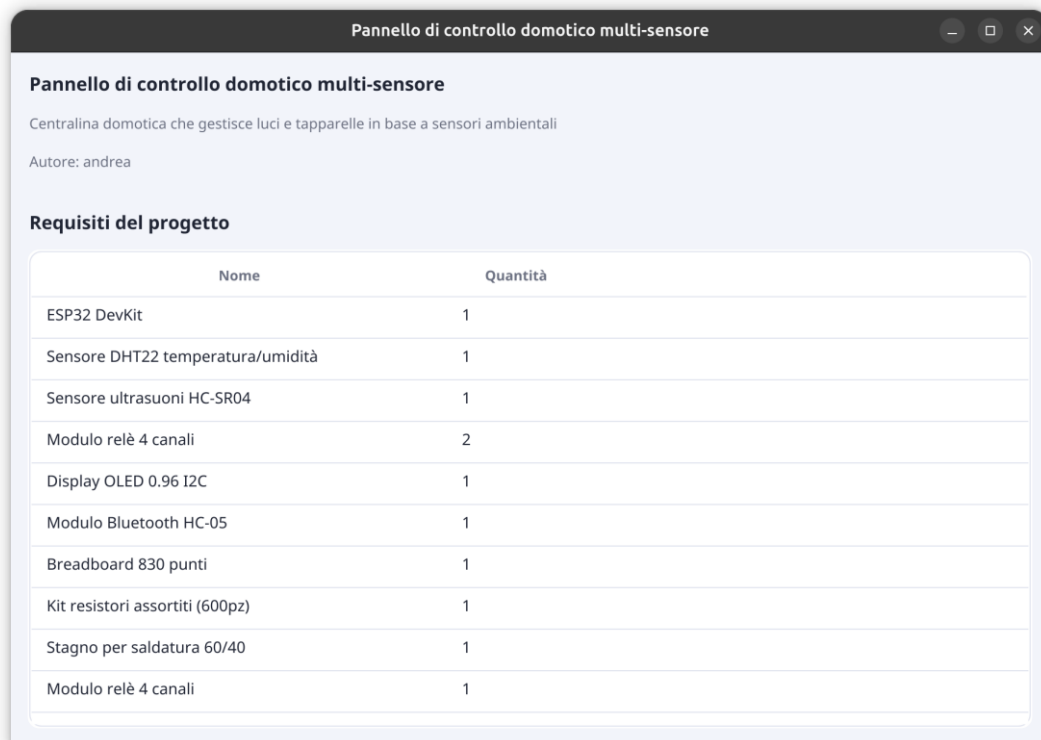
Quantità

10

[Crea e aggiungi all'inventario](#)

5.2. Gestione dei progetti

5.2.1. Visualizzazione di un progetto



Pannello di controllo domotico multi-sensore

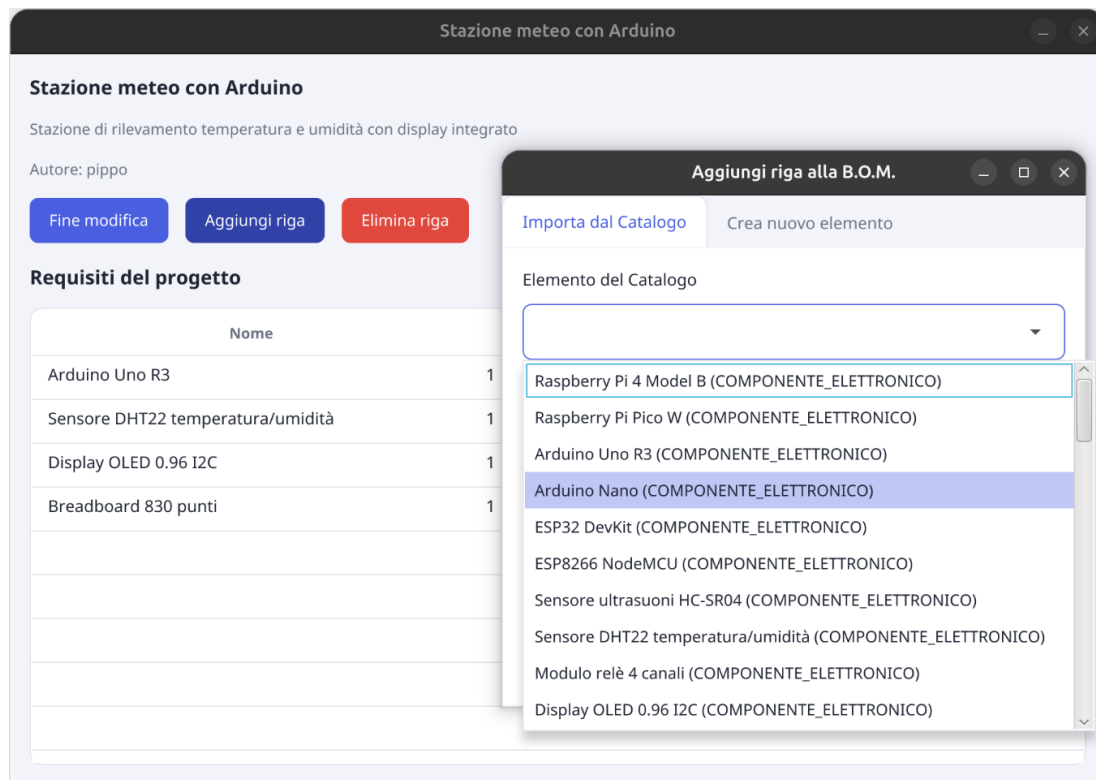
Centralina domotica che gestisce luci e tapparelle in base a sensori ambientali

Autore: andrea

Requisiti del progetto

Nome	Quantità
ESP32 DevKit	1
Sensore DHT22 temperatura/umidità	1
Sensore ultrasuoni HC-SR04	1
Modulo relè 4 canali	2
Display OLED 0.96 I2C	1
Modulo Bluetooth HC-05	1
Breadboard 830 punti	1
Kit resistori assortiti (600pz)	1
Stagno per saldatura 60/40	1
Modulo relè 4 canali	1

5.2.2. Aggiunta di un articolo alla BOM del proprio progetto



Stazione meteo con Arduino

Stazione di rilevamento temperatura e umidità con display integrato

Autore: pippo

[Fine modifica](#) [Aggiungi riga](#) [Elimina riga](#)

Requisiti del progetto

Nome	Quantità
Arduino Uno R3	1
Sensore DHT22 temperatura/umidità	1
Display OLED 0.96 I2C	1
Breadboard 830 punti	1

Aggiungi riga alla B.O.M.

[Importa dal Catalogo](#) [Crea nuovo elemento](#)

Elemento del Catalogo

- Raspberry Pi 4 Model B (COMPONENTE_ELETTRONICO)
- Raspberry Pi Pico W (COMPONENTE_ELETTRONICO)
- Arduino Uno R3 (COMPONENTE_ELETTRONICO)
- Arduino Nano (COMPONENTE_ELETTRONICO)
- ESP32 DevKit (COMPONENTE_ELETTRONICO)
- ESP8266 NodeMCU (COMPONENTE_ELETTRONICO)
- Sensore ultrasuoni HC-SR04 (COMPONENTE_ELETTRONICO)
- Sensore DHT22 temperatura/umidità (COMPONENTE_ELETTRONICO)
- Modulo relè 4 canali (COMPONENTE_ELETTRONICO)
- Display OLED 0.96 I2C (COMPONENTE_ELETTRONICO)

6. Sviluppi futuri

Come detto in precedenza, MakerManager è stato sviluppato con l'obiettivo di essere espandibile in futuro. Attualmente, per esempio, le sottoclassi di ProgettoMaker e ArticoloInventario esistono ma non sono sfruttate nel pieno delle potenzialità.

L'intenzione è quella di aggiungere in futuro della nuova logica di business per:

- Tenere traccia dei costi di un progetto, e cercare nuovi progetti sulla base del budget disponibile o della fascia di prezzo.
- Importare un progetto nella dashboard attivandolo e tenendo traccia dei progressi fatti e avendo a disposizione nuove funzioni per l'utente.
- Interfaccia di prototipazione rapida per il maker, che mette a disposizione nuovi strumenti tra cui un diario personale, l'accesso a nuove tipologie di progetti e la possibilità di aggiungere nuovo contenuto a un progetto (un progetto didattico potrebbe avere una sezione tutorial ad esempio).
- Possibilità di rendere i progetti privati o pubblici a propria scelta.
- Interazione tra gli utenti.

7. Note sull'uso dell'Intelligenza Artificiale

Durante lo sviluppo di **MakerManager** sono stati impiegati strumenti di Intelligenza Artificiale Generativa (modelli usati: Gemini 3.1 Pro e Claude Sonnet 5 con livello di effort medio) con i seguenti scopi:

- **Supporto allo studio e alla scelta delle tecnologie:** chiarimenti su JPA/Hibernate (specialmente nel contesto di classi astratte, sottoclassi ecc), Spring Security, JWT e sulla gestione dell'interfaccia grafica del client.
- **Migliorie estetiche e gestione della GUI** e creazione di una gui temporanea iniziale per effettuare i test (interamente sostituita in seguito).
- **Revisione del codice:** individuazione di bug, falle di sicurezza, ridondanze, incoerenze stilistiche, gestione controllata delle eccezioni.
- **Documentazione:** stesura e revisione della documentazione, diagrammi (mermaid interamente generati da AI), a partire dal codice già scritto.
- **Definizione di alcune strutture base:** in alcuni casi ho utilizzato l'AI per la modellazione dei controller per il client e per la definizione di alcuni modelli, al fine di ottenere una struttura di base da cui partire per l'implementazione.